ĐẠI HỌC PHÚ XUÂN — KHOA CÔNG NGHỆ THÔNG TIN

HƯỚNG DẪN THỰC HÀNH · INT.7.18 — WEB FRONTEND NÂNG CAO

BÀI 10 — THỰC HÀNH

CÂY THÀNH PHẦN, NÂNG TRẠNG THÁI VÀ TÁCH LỚP TRÌNH BÀY

Dự án phu-xuan-react · React 19 + Vite + TypeScript · Node 20 LTS

Hạng mục	Chi tiết
Kèm với	Giáo án lý thuyết Bài 10 — Bài 5 (phần 2)
Thời lượng thực hành tại lớp	60 phút (Lab 1 + Lab 2 + Lab 3)
Thời lượng ở nhà	Lab 4 khoảng 90–120 phút; Lab 5 BONUS 60+ phút
Nhánh Git chính	feature/buoi10-cay-thanh-phan
Nhánh Git từng lab	feature/buoi10-lab1 · feature/buoi10-lab2 · feature/buoi10-lab3 · feature/buoi10-btvn
Chấm bài	Giảng viên clone về, chạy npm install && npm run dev, mở http://localhost:5173, đối chiếu bảng test case ở cuối tài liệu
Nhóm hay cá nhân	Lab 1–4 làm CÁ NHÂN. Lab 5 BONUS làm theo nhóm đồ án đã phân ở Bài 8

MỤC TIÊU THỰC HÀNH BÀI 10

Sau Bài thực hành này, sinh viên sẽ:

1. Chuyển được một state đang đặt sai vị trí lên cha chung gần nhất, và chứng minh bằng React DevTools rằng state chỉ tồn tại đúng một chỗ.
2. Tách một thành phần "gộp" thành cặp Container / View, kiểm chứng bằng cách xem View không có tab hooks trong DevTools.
3. Vận dụng children và named slots để tạo layout dùng lại được ở nhiều trang.
4. Xây dựng được một module tính năng hoàn chỉnh (danh sách di tích Huế) theo đúng ba mẫu vừa học.
5. Trình bày lịch sử commit rõ ràng, mỗi bước là một commit riêng, đẩy lên GitHub và mở Pull Request đúng quy trình.
6. Ghi nhận và xử lý được ít nhất 3 lỗi trong bảng "Lỗi thường gặp" của tài liệu này.

Ánh xạ mục tiêu thực hành với chuẩn đầu ra học phần

CLO	Mục tiêu thực hành phục vụ	Lab đóng góp
CLO1	Vận dụng props, state, hook vào tình huống thực	Lab 1, 2, 4
CLO3	So sánh và chọn mẫu tổ chức thành phần phù hợp	Lab 2, 3, 4, 5
CLO4	Xây dựng giao diện Web nâng cao chạy được, triển khai qua Git	Lab 4, 5
CLO5	Giao tiếp mã nguồn (commit message, PR description), làm việc nhóm	Lab 5 (nhóm) · Yêu cầu PR ở tất cả các lab

YÊU CẦU TRƯỚC KHI BẮT ĐẦU

> **Đọc kỹ trước khi mở VS Code — bỏ qua sẽ mất thời gian debug**
> Nếu bất kỳ mục nào dưới đây chưa đạt, ĐỪNG bắt đầu Lab 1. Quay lại Bài 9 và hoàn thành trước.

1. Trạng thái nhánh Git

Máy của bạn phải có nhánh feature/buoi9-tai-su-dung đã được đẩy lên GitHub và merge vào main. Kiểm tra bằng lệnh:

bash — kiểm tra nhánh hiện tại

```
# Chuyển về thư mục dự án
cd phu-xuan-react

# Cập nhật main mới nhất
git checkout main
git pull origin main

# Xem lịch sử — phải thấy các commit của Bài 9
git log --oneline -5
```

Kết quả mong đợi (5 dòng commit gần nhất, có commit "buoi9: tai su dung component" hoặc tương đương):

Kết quả mong đợi

```
a3c4f21 buoi9: tach card va themcalable
b1d2e88 buoi9: props children cho card
c9e1a02 buoi8: form dat tour co kiem soat
...
```

2. Dự án chạy được

Trước khi vào lab, chạy dự án một lần để chắc không có lỗi tồn đọng từ Bài trước:

bash — chạy dev server

```
npm install      # Chỉ chạy nếu package.json thay đổi
npm run dev

# Mở trình duyệt http://localhost:5173
# Phải thấy trang danh sách 6 tour Huế của Bài 9
```

3. Công cụ đã cài

Công cụ	Kiểm tra bằng lệnh	Phiên bản tối thiểu
Node.js	node -v	v20.x (LTS)
npm	npm -v	10.x
Git	git --version	2.40+
VS Code	code -v (nếu code trong PATH)	Bản ổn định gần nhất

Công cụ	Kiểm tra bằng lệnh	Phiên bản tối thiểu
React Developer Tools	Mở F12, thấy tab Components	Bản mới nhất trên trình duyệt

4. Dữ liệu mẫu

Bài 10 dùng 6 tour Huế của Bài 9 ĐÃ ĐƯỢC MỞ RỘNG. File dữ liệu nằm ở src/data/tours.ts. Nếu file chưa có các trường "price" và "category", cập nhật lại như sau:

TypeScript — src/data/tours.ts (cập nhật đầu Bài 10)

```
export type TourCategory = 'di-tich' | 'am-thuc' | 'thien-nhien' | 'lang-nghe';

export type Tour = {
  id: string;
  name: string;
  category: TourCategory;
  price: number;     // VND
  duration: number;  // giờ
  image: string;
  hot?: boolean;
};

export const tours: Tour[] = [
  { id: 't01', name: 'Đại Nội Huế',       category: 'di-tich',    price: 300000, duration: 3, image: '/images/dai-noi.jpg', hot: true },
  { id: 't02', name: 'Chợ Đông Ba',       category: 'am-thuc',    price: 150000, duration: 2, image: '/images/dong-ba.jpg' },
  { id: 't03', name: 'Cầu Tràng Tiền',    category: 'di-tich',    price: 100000, duration: 1, image: '/images/trang-tien.jpg' },
  { id: 't04', name: 'Lăng Tự Đức',       category: 'di-tich',    price: 400000, duration: 4, image: '/images/tu-duc.jpg', hot: true },
  { id: 't05', name: 'Đầm phá Tam Giang', category: 'thien-nhien',price: 600000, duration: 5, image: '/images/tam-giang.jpg' },
  { id: 't06', name: 'Làng nón Phú Cam',  category: 'lang-nghe',  price: 200000, duration: 2, image: '/images/phu-cam.jpg' },
];
```

Cách tạo nhánh gốc cho Bài 10

Trước khi vào Lab 1, tạo một nhánh mới bao trọn cả Bài:

git checkout -b feature/buoi10-cay-thanh-phan

Sau khi xong mỗi lab, tách nhánh con: git checkout -b feature/buoi10-lab1 (từ nhánh Bài 10).

Cách này giúp có 4 nhánh riêng cho 4 lab in-class + take-home, tiện chấm và tiện revert khi cần.

LAB 1 NÂNG TRẠNG THÁI CHO BỘ LỌC GIÁ

Thời gian: 30 phút | **Mức độ:** CƠ BẢN

Bối cảnh

Ở Bài 9 chúng ta đã có TourListPage hiển thị 6 tour Huế. Bây giờ khách hàng muốn thêm **bộ lọc theo khoảng giá** (min–max). Một số bạn theo bản năng sẽ tạo PriceFilter có state riêng — Lab 1 chỉ ra rằng cách làm này KHÔNG hoạt động, sau đó dạy cách sửa bằng lifting state up.

Kiến trúc mục tiêu sau Lab 1

Cây thành phần

```
TourListPage            <- state minPrice, maxPrice, tất cả 6 tour
 ├── PriceFilter        <- nhận value + onChange, KHÔNG có state riêng
 └── TourGrid           <- nhận filteredTours (đã lọc theo giá)
   └── TourCard × N
```

► Bước 1. Tạo nhánh Lab 1

bash

```
# Đang ở nhánh feature/buoi10-cay-thanh-phan
git checkout -b feature/buoi10-lab1

# Kiểm tra nhánh hiện tại
git branch --show-current
# Kết quả mong đợi: feature/buoi10-lab1
```

► Bước 2. Viết PHIÊN BẢN SAI trước — để tự thấy vấn đề

Trước khi giải đúng, chúng ta viết theo cách "tự nhiên" mà nhiều bạn hay nghĩ ra: PriceFilter tự giữ state. Tạo file:

TypeScript — src/features/tours/PriceFilter.tsx (BẢN SAI)

```
import { useState } from 'react';

export function PriceFilter() {
  // SAI — state chỉ tồn tại trong PriceFilter, TourGrid không thấy được
  const [minPrice, setMinPrice] = useState(0);
  const [maxPrice, setMaxPrice] = useState(1000000);

  return (
    <div className="price-filter">
      <label>
        Từ:
        <input
          type="range"
          min={0}
          max={1000000}
          step={50000}
          value={minPrice}
          onChange={(e) => setMinPrice(Number(e.target.value))}
        />
        <span>{minPrice.toLocaleString()}đ</span>
```

```
    </label>
    <label>
     Đến:
     <input
      type="range"
      min={0}
      max={1000000}
      step={50000}
      value={maxPrice}
      onChange={(e) => setMaxPrice(Number(e.target.value))}
     />
     <span>{maxPrice.toLocaleString()}đ</span>
    </label>
   </div>
  );
}
```

Thêm PriceFilter vào TourListPage (bên trên TourGrid):

TSX — src/pages/TourListPage.tsx (sửa tạm để chèn PriceFilter)

```
import { tours } from '../data/tours';
import { PriceFilter } from '../features/tours/PriceFilter';
import { TourGrid } from '../features/tours/TourGrid';

export function TourListPage() {
 return (
  <main className="tour-list-page">
   <h1>Khám phá Huế qua 6 hành trình</h1>
   <PriceFilter />
   <TourGrid tours={tours} />
  </main>
 );
}
```

Chạy npm run dev, mở trình duyệt, kéo thử thanh trượt.

> **Quan sát — TỰ TRẢ LỜI vào sổ tay trước khi đọc tiếp**
>
> Câu 1: Khi bạn kéo thanh "Từ", giá hiển thị bên cạnh có thay đổi không?
>
> Câu 2: Khi bạn kéo thanh "Từ", danh sách TourGrid có lọc theo không?
>
> Câu 3: Vì sao câu 2 lại như vậy?

► Bước 3. Xem cây thành phần qua React DevTools

Mở F12, chọn tab Components. Bấm vào PriceFilter — bên phải hiện phần "hooks" liệt kê minPrice và maxPrice. Bấm vào TourGrid — không thấy hai giá trị này ở đâu cả. Đây chính là bằng chứng: state đang bị "khóa" trong PriceFilter, TourGrid không có cách nào truy cập được.

React DevTools — điều bạn thấy

```
Components tree:
 ▾ TourListPage
   ▾ PriceFilter
     Hooks:
       1: State: 0        <- minPrice bị khóa trong đây
       2: State: 1000000   <- maxPrice bị khóa trong đây
```

```
▾ TourGrid
  Props:
   tours: Array(6)      <- không có minPrice / maxPrice
```

► Bước 4. Sửa: nâng state lên TourListPage

Bây giờ áp dụng quy tắc 5 bước từ giáo án lý thuyết. Cha chung gần nhất của PriceFilter và TourGrid là TourListPage. Nâng state minPrice và maxPrice lên đó, truyền value + onChange xuống PriceFilter, truyền tours đã lọc xuống TourGrid.

TSX — src/features/tours/PriceFilter.tsx (BẢN ĐÚNG)

```tsx
type PriceFilterProps = {
  minPrice: number;
  maxPrice: number;
  onMinChange: (value: number) => void;
  onMaxChange: (value: number) => void;
};

// PriceFilter giờ là "presentational" — không có useState, chỉ nhận props và vẽ
export function PriceFilter({ minPrice, maxPrice, onMinChange, onMaxChange }: PriceFilterProps) {
  return (
    <div className="price-filter">
      <label>
        Từ:
        <input
          type="range"
          min={0}
          max={1000000}
          step={50000}
          value={minPrice}
          onChange={(e) => onMinChange(Number(e.target.value))}
        />
        <span>{minPrice.toLocaleString()}đ</span>
      </label>
      <label>
        Đến:
        <input
          type="range"
          min={0}
          max={1000000}
          step={50000}
          value={maxPrice}
          onChange={(e) => onMaxChange(Number(e.target.value))}
        />
        <span>{maxPrice.toLocaleString()}đ</span>
      </label>
    </div>
  );
}
```

TSX — src/pages/TourListPage.tsx (BẢN ĐÚNG)

```tsx
import { useState } from 'react';
import { tours } from '../data/tours';
import { PriceFilter } from '../features/tours/PriceFilter';
import { TourGrid } from '../features/tours/TourGrid';
```

```
export function TourListPage() {
 // State giờ nằm ở cha chung — single source of truth
 const [minPrice, setMinPrice] = useState(0);
 const [maxPrice, setMaxPrice] = useState(1000000);

 // Lọc tours theo khoảng giá — tính lại mỗi lần state đổi
 const filteredTours = tours.filter(
  (t) => t.price >= minPrice && t.price <= maxPrice
 );

 return (
  <main className="tour-list-page">
   <h1>Khám phá Huế qua 6 hành trình</h1>
   <PriceFilter
    minPrice={minPrice}
    maxPrice={maxPrice}
    onMinChange={setMinPrice}
    onMaxChange={setMaxPrice}
   />
   <p className="filter-summary">
    Đang hiển thị {filteredTours.length} / {tours.length} tour
   </p>
   <TourGrid tours={filteredTours} />
  </main>
 );
}
```

► Bước 5. Kiểm chứng qua trình duyệt

Chạy npm run dev (nếu chưa chạy). Kéo thử các ngưỡng sau, danh sách phải phản ứng đúng:

Kéo min =	Kéo max =	Số tour hiển thị	Các tour thấy được
0	1.000.000	6	Toàn bộ
100.000	300.000	3	Đại Nội (300k), Chợ Đông Ba (150k), Cầu Tràng Tiền (100k)
500.000	1.000.000	1	Đầm phá Tam Giang (600k)
700.000	1.000.000	0	Không có — hiển thị "0 / 6 tour"

► Bước 6. Xem lại React DevTools sau khi sửa

React DevTools — sau khi lifting

```
Components tree:
  ▾ TourListPage
    Hooks:
      1: State: 0        <- minPrice giờ nằm ở ĐÂY
      2: State: 1000000   <- maxPrice giờ nằm ở ĐÂY
    ▾ PriceFilter
      Props:
        minPrice: 0
        maxPrice: 1000000
        onMinChange: f()
```

```
    onMaxChange: f()
  Hooks:          <- KHÔNG CÓ hooks nữa
▾ TourGrid
  Props:
   tours: Array(6)   <- tours đã được lọc từ TourListPage
```

► Bước 7. Commit với message mô tả rõ ràng

bash

```
git add .
git status  # xem lại các file đã đổi
git commit -m "buoi10-lab1: nang state gia len TourListPage (single source of truth)"
git push origin feature/buoi10-lab1
```

CHECKPOINT — Lab 1

Chỉ chuyển sang lab tiếp theo khi toàn bộ các mục dưới đây đạt:

[ ] Kéo thanh min hoặc max, danh sách TourGrid lọc lại đúng theo bảng test case ở bước 5.

[ ] React DevTools cho thấy TourListPage có 2 state (minPrice, maxPrice); PriceFilter không có state nào (mục Hooks trống).

[ ] Không còn bản sao state ở PriceFilter — grep "useState" trong PriceFilter.tsx phải không tìm thấy.

[ ] Đã commit đúng message quy chuẩn và push lên nhánh feature/buoi10-lab1 trên GitHub.

Bảng lỗi thường gặp — Lab 1

Vấn đề gặp phải	Nguyên nhân có thể	Cách khắc phục
Kéo slider nhưng số bên cạnh không đổi.	Quên gọi setMinPrice(Number(e.target.value)) — vẫn để callback rỗng.	Kiểm tra hàm onChange của input phải gọi setter và ép Number.
TypeScript đỏ: "Property onMinChange does not exist"	Chưa cập nhật PriceFilterProps hoặc chưa xóa useState cũ.	Bảo đảm PriceFilter đã dùng đúng type mới; xóa hoàn toàn 2 dòng useState cũ.
Không tìm thấy component PriceFilter trong DevTools	PriceFilter chưa được import ở TourListPage.	Thêm import { PriceFilter } from "../features/tours/PriceFilter" ở đầu file.
Số tour hiển thị ra 6 dù đã kéo min lên 500k.	Lọc bằng "t.price > minPrice" (sai toán tử) hoặc quên truyền tours đã lọc.	Dùng >= và <=; truyền filteredTours xuống TourGrid, không phải tours gốc.
State hai slider "nhảy" ngẫu nhiên qua lại.	Truyền onMinChange = setMaxPrice (nhầm setter).	Đọc lại phần "props truyền xuống" ở TourListPage, mỗi setter dành cho đúng thanh.

LAB 2 TÁCH TOURLISTPAGE THÀNH CONTAINER + VIEW

Thời gian: 20 phút | **Mức độ:** CƠ BẢN — TRUNG

Bối cảnh

Sau Lab 1, TourListPage đang **vừa quản lý state, vừa vẽ giao diện**, vừa lọc dữ liệu. Ba việc trong một file 40 dòng thì chưa nặng, nhưng khi Bài 17 chúng ta thêm useEffect + fetch API, file sẽ vượt 150 dòng. Lab 2 tách sẵn hai lớp để Bài 17 chỉ việc gắn API vào.

Kiến trúc mục tiêu sau Lab 2

Cây thành phần

```
App
└── TourListContainer      <- state, logic lọc, sau này có useEffect fetch
    └── TourListView       <- không có state, chỉ nhận props và vẽ
        ├── PriceFilter
        ├── FilterSummary
        └── TourGrid
            └── TourCard × N
```

► Bước 1. Tạo nhánh Lab 2

bash

```
# Đang trên feature/buoi10-lab1 và Lab 1 đã commit xong
git checkout feature/buoi10-cay-thanh-phan
git merge feature/buoi10-lab1      # gộp kết quả Lab 1
git checkout -b feature/buoi10-lab2
```

► Bước 2. Tạo TourListView — thành phần trình bày

View chỉ có props và JSX, không có bất kỳ hook nào ngoại trừ kiểu React.ReactNode nếu cần.

TypeScript — src/features/tours/TourListView.tsx (MỚI)

```
import type { Tour } from '../../data/tours';
import { PriceFilter } from './PriceFilter';
import { TourGrid } from './TourGrid';

type TourListViewProps = {
  // Dữ liệu đã lọc, sẵn sàng hiển thị
  filteredTours: Tour[];
  totalCount: number;
  // Trạng thái bộ lọc
  minPrice: number;
  maxPrice: number;
  // Callback gửi ngược lên
  onMinChange: (value: number) => void;
  onMaxChange: (value: number) => void;
};

// View chỉ có props và JSX — không state, không effect, không fetch
export function TourListView({
  filteredTours, totalCount, minPrice, maxPrice, onMinChange, onMaxChange,
}: TourListViewProps) {
  return (
```

```
  <main className="tour-list-page">
    <h1>Khám phá Huế qua 6 hành trình</h1>
    <PriceFilter
      minPrice={minPrice}
      maxPrice={maxPrice}
      onMinChange={onMinChange}
      onMaxChange={onMaxChange}
    />
    <p className="filter-summary">
      Đang hiển thị {filteredTours.length} / {totalCount} tour
    </p>
    {filteredTours.length === 0 ? (
      <p className="empty-state">
        Không có tour nào phù hợp với khoảng giá này. Hãy nới rộng thanh trượt.
      </p>
    ) : (
      <TourGrid tours={filteredTours} />
    )}
  </main>
 );
}
```

► Bước 3. Tạo TourListContainer — thành phần chứa dữ liệu

TypeScript — src/features/tours/TourListContainer.tsx (MỚI)

```
import { useState } from 'react';
import { tours } from '../../data/tours';
import { TourListView } from './TourListView';

// Container chịu trách nhiệm state + logic; không đụng đến JSX của trang
export function TourListContainer() {
 const [minPrice, setMinPrice] = useState(0);
 const [maxPrice, setMaxPrice] = useState(1000000);

 const filteredTours = tours.filter(
   (t) => t.price >= minPrice && t.price <= maxPrice
 );

 return (
   <TourListView
     filteredTours={filteredTours}
     totalCount={tours.length}
     minPrice={minPrice}
     maxPrice={maxPrice}
     onMinChange={setMinPrice}
     onMaxChange={setMaxPrice}
   />
 );
}
```

► Bước 4. Xoá phiên bản cũ của TourListPage và cập nhật App.tsx

bash

```
# Xoá file cũ
rm src/pages/TourListPage.tsx
```

```
# rmdir nếu thư mục pages rỗng
rmdir src/pages 2>/dev/null || true
```

TSX — src/App.tsx (cập nhật)

```
import { TourListContainer } from './features/tours/TourListContainer';

function App() {
  return <TourListContainer />;
}

export default App;
```

► Bước 5. Kiểm chứng: TourListView phải không có tab hooks

Đây là "chứng chỉ" cho biết bạn đã tách đúng. Mở F12, chọn tab Components, bấm vào TourListView — bên phải KHÔNG được có mục "Hooks". Nếu vẫn có, quay lại kiểm tra: bạn có lỡ để lại useState nào ở View không?

React DevTools — sau khi tách

```
Components tree:
  ▾ TourListContainer
    Hooks:
      1: State: 0
      2: State: 1000000
    ▾ TourListView
      Props: (nhiều)
      Hooks:          <- MỤC NÀY PHẢI KHÔNG XUẤT HIỆN
      ▾ PriceFilter
      ▾ TourGrid
```

► Bước 6. Chạy thử — chức năng phải giống hệt Lab 1

Một dấu hiệu tách đúng: refactor không làm đổi hành vi. Kéo thanh min/max, danh sách phải lọc y hệt bảng test case ở Lab 1. Nếu chức năng đổi, khả năng cao bạn đã bỏ sót một prop khi truyền từ Container xuống View.

► Bước 7. Commit

bash

```
git add .
git commit -m "buoi10-lab2: tach TourListContainer va TourListView (presentational vs container)"
git push origin feature/buoi10-lab2
```

CHECKPOINT — Lab 2

Chỉ chuyển sang lab tiếp theo khi toàn bộ các mục dưới đây đạt:

- [ ] Đã có 2 file mới: TourListContainer.tsx và TourListView.tsx.
- [ ] File TourListPage.tsx cũ đã bị xóa (grep tìm "TourListPage" trong src không còn kết quả).
- [ ] Chức năng lọc giá vẫn chạy y hệt như Lab 1 — chọn 100k–300k phải thấy đúng 3 tour.
- [ ] React DevTools: TourListView không hiện tab "Hooks" khi bấm vào.

- [ ] Đã commit với message quy chuẩn và push lên GitHub.

Bảng lỗi thường gặp — Lab 2

Vấn đề gặp phải	Nguyên nhân có thể	Cách khắc phục
TypeScript đỏ: "Cannot find name Tour"	Chưa import kiểu Tour từ src/data/tours.ts.	Thêm: import type { Tour } from "../../data/tours".
View vẫn hiện mục Hooks trong DevTools.	Còn sót useState (thường là thanh loading hoặc UI state).	Xem lại View, xóa hết useState. State thuộc về Container.
App bị lỗi trắng trang, console báo "Container undefined".	Đường dẫn import sai (../ hay ../../).	Kiểm tra cấu trúc thư mục: src/App.tsx dùng ./features/tours/...
Chức năng lọc sai sau khi tách.	Truyền tours gốc thay vì filteredTours xuống View.	Xem lại Container, prop filteredTours phải là kết quả của .filter().
Console báo "React key warning" trong TourGrid.	Không phải lỗi của Lab 2 — do TourGrid Bài 9 dùng index làm key.	Sửa TourGrid: key={tour.id} thay vì key={index} — coi như bonus fix.

LAB 3 COMPOSITION VÀ THÊM SEARCHBOX ANH EM

Thời gian: 15 phút | **Mức độ:** TRUNG

Bối cảnh

Bây giờ trang cần một ô tìm kiếm theo tên tour. SearchBox và TourGrid lại là hai anh em — cùng cần biết "từ khóa đang gõ". Đây là lần thứ hai chúng ta lifting state — nhưng lần này thêm một layer nữa: PageLayout với named slots để tách phần khung khỏi phần nội dung, phục vụ tái sử dụng ở các trang khác.

Kiến trúc mục tiêu sau Lab 3

Cây thành phần

```
TourListContainer          <- state: minPrice, maxPrice, query
  └── PageLayout           <- khung dùng chung: header + main + sidebar
    ├── SearchBox (slot: header)
    ├── PriceFilter (slot: sidebar)
    └── TourGrid (slot: main)
```

► Bước 1. Tạo nhánh Lab 3

bash

```
git checkout feature/buoi10-cay-thanh-phan
git merge feature/buoi10-lab2
git checkout -b feature/buoi10-lab3
```

► Bước 2. Tạo PageLayout với 3 khe có tên

TypeScript — src/components/PageLayout.tsx

```
import type { ReactNode } from 'react';

type PageLayoutProps = {
  header: ReactNode;
  sidebar: ReactNode;
  main: ReactNode;
};

// Khung 3 khe, dùng lại cho mọi trang danh sách (tour, nhà hàng, di tích...)
export function PageLayout({ header, sidebar, main }: PageLayoutProps) {
  return (
    <div className="page-layout">
      <header className="page-layout__header">{header}</header>
      <div className="page-layout__body">
        <aside className="page-layout__sidebar">{sidebar}</aside>
        <main className="page-layout__main">{main}</main>
      </div>
    </div>
  );
}
```

► Bước 3. Tạo SearchBox — presentational thuần

TypeScript — src/features/tours/SearchBox.tsx

```
type SearchBoxProps = {
```

```
  value: string;
  onChange: (value: string) => void;
  placeholder?: string;
};

// Không có state riêng — luôn nhận value + onChange từ ngoài
export function SearchBox({ value, onChange, placeholder }: SearchBoxProps) {
  return (
    <div className="search-box">
      <input
        type="search"
        value={value}
        onChange={(e) => onChange(e.target.value)}
        placeholder={placeholder ?? 'Tìm kiếm...'}
      />
      {value && (
        <button
          type="button"
          className="search-box__clear"
          onClick={() => onChange('')}
          aria-label="Xóa từ khóa"
        >
          X
        </button>
      )}
    </div>
  );
}
```

► Bước 4. Cập nhật Container — thêm query, gộp bộ lọc

TypeScript — src/features/tours/TourListContainer.tsx (cập nhật)

```
import { useState } from 'react';
import { tours } from '../../data/tours';
import { TourListView } from './TourListView';

export function TourListContainer() {
  const [minPrice, setMinPrice] = useState(0);
  const [maxPrice, setMaxPrice] = useState(1000000);
  const [query, setQuery] = useState('');

  // Lọc kết hợp: theo giá VÀ theo từ khóa
  const normalized = query.trim().toLowerCase();
  const filteredTours = tours
    .filter((t) => t.price >= minPrice && t.price <= maxPrice)
    .filter((t) =>
      normalized === '' ? true : t.name.toLowerCase().includes(normalized)
    );

  return (
    <TourListView
      filteredTours={filteredTours}
      totalCount={tours.length}
      minPrice={minPrice}
      maxPrice={maxPrice}
      query={query}
```

```
    onMinChange={setMinPrice}
    onMaxChange={setMaxPrice}
    onQueryChange={setQuery}
   />
  );
}
```

► Bước 5. Cập nhật View — dùng PageLayout với 3 khe

TypeScript — src/features/tours/TourListView.tsx (cập nhật)

```
import type { Tour } from '../../data/tours';
import { PageLayout } from '../../components/PageLayout';
import { PriceFilter } from './PriceFilter';
import { SearchBox } from './SearchBox';
import { TourGrid } from './TourGrid';

type TourListViewProps = {
  filteredTours: Tour[];
  totalCount: number;
  minPrice: number;
  maxPrice: number;
  query: string;
  onMinChange: (value: number) => void;
  onMaxChange: (value: number) => void;
  onQueryChange: (value: string) => void;
};

export function TourListView({
  filteredTours, totalCount, minPrice, maxPrice, query,
  onMinChange, onMaxChange, onQueryChange,
}: TourListViewProps) {
  return (
    <PageLayout
      header={
        <>
          <h1>Khám phá Huế qua {totalCount} hành trình</h1>
          <SearchBox
            value={query}
            onChange={onQueryChange}
            placeholder="Tìm tên tour (VD: Đại Nội)..."
          />
        </>
      }
      sidebar={
        <>
          <h2>Bộ lọc</h2>
          <PriceFilter
            minPrice={minPrice}
            maxPrice={maxPrice}
            onMinChange={onMinChange}
            onMaxChange={onMaxChange}
          />
        </>
      }
      main={
        <>
```

```
        <p className="filter-summary">
         {filteredTours.length} / {totalCount} tour phù hợp
        </p>
        {filteredTours.length === 0 ? (
         <p className="empty-state">
          Không có tour nào phù hợp. Hãy nới bộ lọc hoặc thử từ khóa khác.
         </p>
        ) : (
         <TourGrid tours={filteredTours} />
        )}
       </>
     }
    />
  );
}
```

► Bước 6. Kiểm chứng bảng test case

Kịch bản	Thao tác	Kết quả mong đợi
Lọc giá + từ khóa cùng chạy	Kéo min=100k max=500k, gõ "Đại"	Chỉ Đại Nội Huế hiện lên (1 tour)
Xóa từ khóa bằng nút X	Gõ "Chợ", sau đó bấm nút X	Ô tìm kiếm trống, danh sách trở về theo bộ lọc giá
Không phân biệt hoa/thường	Gõ "TAM giang"	Đầm phá Tam Giang xuất hiện
Empty state	Gõ "sushi"	Hiển thị "Không có tour nào phù hợp..."
Layout dùng lại được	Xem file PageLayout.tsx	Không hardcode bất cứ nội dung tour nào — có thể tái dùng cho trang khác

► Bước 7. Commit

```bash
git add .
git commit -m "buoi10-lab3: them SearchBox va PageLayout composition (3 named slots)"
git push origin feature/buoi10-lab3
```

CHECKPOINT — Lab 3

Chỉ chuyển sang lab tiếp theo khi toàn bộ các mục dưới đây đạt:

[ ] Có PageLayout.tsx với 3 slot: header, sidebar, main; PageLayout không import bất cứ gì thuộc feature/tours.

[ ] SearchBox không có useState — chỉ nhận value và onChange.

[ ] Container quản lý 3 state: minPrice, maxPrice, query. View không có state nào.

[ ] Bảng test case ở bước 6 — cả 5 kịch bản đều đạt.

[ ] Đã đẩy nhánh feature/buoi10-lab3 lên GitHub.

LAB 4 XÂY DỰNG MODULE DANH SÁCH DI TÍCH HUẾ

Thời gian: 90–120 phút (làm ở nhà) | **Mức độ:** NÂNG CAO

Đây là bài tập về nhà bắt buộc

Hạn nộp: 23:59 ngày trước Bài 11.

Cách nộp: đẩy nhánh feature/buoi10-btvn lên GitHub và mở Pull Request vào main, cc @cuong.nguyen.

Điểm: chấm theo rubric ở cuối Lab. Đạt trung bình = 5/10; xuất sắc = 9–10 nếu có phần "mở rộng".

Chính sách AI: cho phép hỏi Claude/Copilot, nhưng ghi vào README phần nào có sự trợ giúp — nếu vấn đáp không giải thích được, trừ điểm cá nhân.

Bối cảnh

Sau khi có module tour, phu-xuan-react cần thêm module "Di tích Huế" phục vụ trang thông tin văn hóa. Module này về mặt cấu trúc giống module tour nhưng có thêm 2 thử thách: (1) dùng custom hook thay cho container, (2) thêm bộ lọc theo trạng thái (đã tham quan / chưa).

Kiến trúc mục tiêu

Cây thư mục sau Lab 4

```
src/
├── data/
│   └── ditich.ts              (bạn tự tạo — dữ liệu 8 di tích Huế)
└── features/
    └── ditich/
        ├── types.ts           (kiểu DiTich, DiTichFilter)
        ├── useDiTichList.ts   (custom hook — thay cho container)
        ├── DiTichListView.tsx (presentational)
        ├── DiTichCard.tsx     (presentational)
        ├── DiTichFilter.tsx   (presentational)
        └── DiTichSearchBox.tsx (presentational — có thể tái sử dụng SearchBox chung)
```

► Bước 1. Chuẩn bị dữ liệu — 8 di tích Huế

Tạo file src/data/ditich.ts với ít nhất 8 di tích thật. Bảng gợi ý (bạn được phép thêm/thay):

Tên	Loại	Thế kỷ xây dựng	Đã tham quan mặc định
Đại Nội Huế	kinh-thanh		false
Lăng Tự Đức	lang-tam		false
Lăng Khải Định	lang-tam		false
Lăng Minh Mạng	lang-tam		false
Chùa Thiên Mụ	chua		true
Cầu Trường Tiền	cong-trinh-cong-cong		true

Tên	Loại	Thế kỷ xây dựng	Đã tham quan mặc định
Đàn Nam Giao	te-dan		false
Chùa Từ Đàm	chua		false

TypeScript — src/features/ditich/types.ts (bạn tự tạo)

```
export type LoaiDiTich = 'kinh-thanh' | 'lang-tam' | 'chua' | 'te-dan' | 'cong-trinh-cong-cong';

export type DiTich = {
  id: string;
  ten: string;
  loai: LoaiDiTich;
  theKy: number;
  moTa: string;
  daThamQuan: boolean;
};

export type DiTichFilter = {
  tuKhoa: string;
  loai: LoaiDiTich | 'tat-ca';
  chiChuaThamQuan: boolean;
};
```

► Bước 2. Viết custom hook useDiTichList

Custom hook là biến thể hiện đại của container: gom logic vào một hàm useXxx() có thể được gọi ở bất kỳ view nào. Yêu cầu:

- Hook nhận vào danh sách di tích gốc và trả về { danhSachHienThi, boLoc, capNhatBoLoc, danhDauDaThamQuan, resetBoLoc }.
- danhSachHienThi phải được tính lại tự động khi bất kỳ trường nào của boLoc thay đổi.
- danhDauDaThamQuan(id: string) đảo trạng thái daThamQuan của di tích có id tương ứng.
- resetBoLoc đưa boLoc về mặc định (từ khóa rỗng, loại = "tat-ca", chiChuaThamQuan = false).

TypeScript — src/features/ditich/useDiTichList.ts (khung — bạn hoàn thiện)

```
import { useState } from 'react';
import type { DiTich, DiTichFilter } from './types';

const BO_LOC_MAC_DINH: DiTichFilter = {
  tuKhoa: '',
  loai: 'tat-ca',
  chiChuaThamQuan: false,
};

export function useDiTichList(dataGoc: DiTich[]) {
  const [danhSach, setDanhSach] = useState<DiTich[]>(dataGoc);
  const [boLoc, setBoLoc] = useState<DiTichFilter>(BO_LOC_MAC_DINH);

  // 1. Tính danhSachHienThi từ danhSach + boLoc
  const tuKhoaNormalized = boLoc.tuKhoa.trim().toLowerCase();
  const danhSachHienThi = danhSach.filter((d) => {
    // TODO 1: lọc theo tuKhoa (khớp ten không phân biệt hoa/thường)
```

```
  // TODO 2: lọc theo loai (nếu 'tat-ca' thì cho qua tất cả)
  // TODO 3: lọc theo chiChuaThamQuan (nếu true thì chỉ giữ d.daThamQuan === false)
  return true; // xoá dòng này khi hoàn thành các TODO
 });

 // 2. Hàm cập nhật bộ lọc — nhận partial để cập nhật một phần
 const capNhatBoLoc = (patch: Partial<DiTichFilter>) => {
  setBoLoc((prev) => ({ ...prev, ...patch }));
 };

 // 3. Đảo trạng thái tham quan của một di tích
 const danhDauDaThamQuan = (id: string) => {
  // TODO 4: dùng setDanhSach với hàm updater, .map qua danh sách
  // Nếu d.id === id thì đảo d.daThamQuan
  // Nếu không thì giữ nguyên
 };

 const resetBoLoc = () => setBoLoc(BO_LOC_MAC_DINH);

 return { danhSachHienThi, boLoc, capNhatBoLoc, danhDauDaThamQuan, resetBoLoc };
}
```

Gợi ý: mẫu cập nhật state không đột biến (immutable)

setDanhSach((prev) => prev.map((d) => d.id === id ? { ...d, daThamQuan: !d.daThamQuan } : d))

Đọc thêm ở giáo án lý thuyết Bài 6 mục "hook và immutability".

► Bước 3. Viết các presentational component

Tạo 4 file. Yêu cầu chung: KHÔNG được có useState, useEffect, useRef trong bất kỳ file view nào. Nếu bạn thấy cần, đó là dấu hiệu logic đó phải chuyển vào hook.

TypeScript — src/features/ditich/DiTichCard.tsx

```
import type { DiTich } from './types';

type DiTichCardProps = {
 diTich: DiTich;
 onToggleThamQuan: (id: string) => void;
};

export function DiTichCard({ diTich, onToggleThamQuan }: DiTichCardProps) {
 return (
  <article className={`ditich-card ${diTich.daThamQuan ? 'ditich-card--visited' : ''}`}>
   <h3>{diTich.ten}</h3>
   <p className="ditich-card__meta">
    {diTich.loai.replace(/-/g, ' ')} — thế kỷ {diTich.theKy}
   </p>
   <p className="ditich-card__desc">{diTich.moTa}</p>
   <button
    type="button"
    onClick={() => onToggleThamQuan(diTich.id)}
   >
    {diTich.daThamQuan ? 'Bỏ đánh dấu' : 'Đã tham quan'}
   </button>
  </article>
```

```
  );
}
```

TypeScript — src/features/ditich/DiTichFilter.tsx

```
import type { DiTichFilter as FilterState, LoaiDiTich } from './types';

type DiTichFilterProps = {
  boLoc: FilterState;
  onChange: (patch: Partial<FilterState>) => void;
  onReset: () => void;
};

const CAC_LOAI: Array<{ value: LoaiDiTich | 'tat-ca'; label: string }> = [
  { value: 'tat-ca',              label: 'Tất cả' },
  { value: 'kinh-thanh',          label: 'Kinh thành' },
  { value: 'lang-tam',            label: 'Lăng tẩm' },
  { value: 'chua',                label: 'Chùa' },
  { value: 'te-dan',              label: 'Tế đàn' },
  { value: 'cong-trinh-cong-cong',  label: 'Công trình công cộng' },
];

export function DiTichFilter({ boLoc, onChange, onReset }: DiTichFilterProps) {
  return (
    <div className="ditich-filter">
      <label>
        Loại di tích:
        <select
          value={boLoc.loai}
          onChange={(e) => onChange({ loai: e.target.value as LoaiDiTich | 'tat-ca' })}
        >
          {CAC_LOAI.map((l) => (
            <option key={l.value} value={l.value}>{l.label}</option>
          ))}
        </select>
      </label>
      <label>
        <input
          type="checkbox"
          checked={boLoc.chiChuaThamQuan}
          onChange={(e) => onChange({ chiChuaThamQuan: e.target.checked })}
        />
        Chỉ hiện những nơi chưa tham quan
      </label>
      <button type="button" onClick={onReset}>Đặt lại bộ lọc</button>
    </div>
  );
}
```

► Bước 4. Ghép View — DiTichListView

TypeScript — src/features/ditich/DiTichListView.tsx

```
import { useDiTichList } from './useDiTichList';
import { danhSachDiTich } from '../../data/ditich';
import { DiTichCard } from './DiTichCard';
import { DiTichFilter } from './DiTichFilter';
import { SearchBox } from '../tours/SearchBox'; // tái sử dụng SearchBox từ Lab 3
```

```
import { PageLayout } from '../../components/PageLayout';

// View gọi hook trực tiếp — không cần Container nữa
export function DiTichListView() {
 const { danhSachHienThi, boLoc, capNhatBoLoc, danhDauDaThamQuan, resetBoLoc } =
  useDiTichList(danhSachDiTich);

 return (
  <PageLayout
   header={
    <>
     <h1>Di tích Huế</h1>
     <SearchBox
      value={boLoc.tuKhoa}
      onChange={(value) => capNhatBoLoc({ tuKhoa: value })}
      placeholder="Tìm di tích..."
     />
    </>
   }
   sidebar={
    <DiTichFilter
     boLoc={boLoc}
     onChange={capNhatBoLoc}
     onReset={resetBoLoc}
    />
   }
   main={
    <>
     <p>Hiển thị {danhSachHienThi.length} / {danhSachDiTich.length} di tích</p>
     <div className="ditich-grid">
      {danhSachHienThi.map((d) => (
       <DiTichCard
        key={d.id}
        diTich={d}
        onToggleThamQuan={danhDauDaThamQuan}
       />
      ))}
     </div>
    </>
   }
  />
 );
}
```

► Bước 5. Thêm route đơn giản trong App.tsx

Chưa dùng React Router (Bài 13) — tạm thời dùng useState để chuyển tab:

TSX — src/App.tsx (cập nhật)

```
import { useState } from 'react';
import { TourListContainer } from './features/tours/TourListContainer';
import { DiTichListView } from './features/ditich/DiTichListView';

type Trang = 'tour' | 'ditich';

function App() {
```

```
  const [trang, setTrang] = useState<Trang>('tour');
  return (
    <>
      <nav className="top-nav">
        <button onClick={() => setTrang('tour')} disabled={trang === 'tour'}>Tour</button>
        <button onClick={() => setTrang('ditich')} disabled={trang === 'ditich'}>Di tích</button>
      </nav>
      {trang === 'tour' ? <TourListContainer /> : <DiTichListView />}
    </>
  );
}

export default App;
```

► Bước 6. Kiểm chứng theo bảng test case đầy đủ

#	Thao tác	Kết quả mong đợi
1	Bấm tab "Di tích"	Trang chuyển sang danh sách 8 di tích Huế
2	Gõ "lăng" vào ô tìm kiếm	Còn 3 di tích: Lăng Tự Đức, Lăng Khải Định, Lăng Minh Mạng
3	Chọn loại "Chùa"	Hiện Chùa Thiên Mụ, Chùa Từ Đàm (bỏ qua từ khóa)
4	Bấm "Đã tham quan" trên Đại Nội	Thẻ Đại Nội đổi style (class ditich-card--visited); nút đổi thành "Bỏ đánh dấu"
5	Bật checkbox "chỉ chưa tham quan"	Đại Nội (vừa đánh dấu ở bước 4) biến mất
6	Bấm "Đặt lại bộ lọc"	Ô tìm kiếm trống, dropdown về "Tất cả", checkbox tắt
7	Chuyển sang tab "Tour" rồi quay lại "Di tích"	Trạng thái "Đại Nội đã tham quan" phải MẤT (do component unmount) — đây là hạn chế sẽ giải quyết ở Bài 12 (Context)
8	Mở React DevTools, bấm vào DiTichListView	Hooks chỉ có duy nhất 1 hook: useDiTichList (custom); không thấy useState/useEffect trực tiếp trong View

► Bước 7. Viết README ngắn — tài liệu bắt buộc

Tạo file docs/buoi10-btvn.md ghi lại:

- Sơ đồ cây thành phần của trang Di tích (vẽ tay chụp ảnh, hoặc dùng excalidraw.com xuất PNG).
- Danh sách 3 mẫu đã áp dụng, mỗi mẫu 2–3 câu giải thích tại sao chọn.
- Phần "được AI hỗ trợ": khai báo trung thực đoạn nào bạn nhờ Claude/Copilot viết. Nếu KHÔNG dùng AI, ghi rõ "Không dùng AI".
- Danh sách khó khăn đã gặp và cách khắc phục (tối thiểu 2 mục).

► Bước 8. Commit và mở Pull Request

bash

```
git checkout feature/buoi10-cay-thanh-phan
git merge feature/buoi10-lab3
git checkout -b feature/buoi10-btvn

# ... làm Lab 4 ở nhánh này ...
```

```
git add .
git commit -m "buoi10-btvn: module di tich Hue voi custom hook useDiTichList"
git push origin feature/buoi10-btvn

# Mở trình duyệt tới trang GitHub của repo, bấm "Compare & pull request"
# Tiêu đề PR: "Bài 10 BTVN — Module di tích Huế"
# Mô tả PR: dán liên kết docs/buoi10-btvn.md, gắn label "buoi10", cc @cuong.nguyen
```

CHECKPOINT — Lab 4

Chỉ chuyển sang lab tiếp theo khi toàn bộ các mục dưới đây đạt:

- [ ] Đủ 6 file như cấu trúc thư mục ở bước đầu bài.
- [ ] useDiTichList hoàn thiện đủ 4 TODO — chạy đúng 8 kịch bản trong bảng test case.
- [ ] DiTichListView không có useState/useEffect trực tiếp (chỉ gọi useDiTichList).
- [ ] Đã tái sử dụng SearchBox và PageLayout của Lab 3 — grep cho thấy 2 chỗ import.
- [ ] File docs/buoi10-btvn.md có đủ: sơ đồ cây, giải thích 3 mẫu, khai báo AI, khó khăn.
- [ ] PR đã mở, có ít nhất 3 commit tách biệt (không dồn 1 commit khổng lồ).

Rubric chấm Lab 4 (thang 10)

Tiêu chí	Trọng số	Đạt (7–10)	Chưa đạt (0–4)
Chức năng đúng	30%	8/8 kịch bản test case đạt.	Dưới 5/8 kịch bản đạt hoặc app không chạy.
Kiến trúc tách rõ	25%	View không có useState/useEffect trực tiếp; hook đúng chuẩn; presentational không import fetch.	View vẫn còn state/effect; hook không tách được logic khỏi UI.
TypeScript sạch	15%	Không có "any"; các prop có kiểu rõ ràng; npm run build không có warning.	Nhiều "any" hoặc bỏ tắt strict; build có warning về type.
Lịch sử commit	10%	≥3 commit tách biệt, message theo mẫu buoi10-btvn: <hành động>.	Chỉ 1 commit "final" hoặc message không có ý nghĩa.
Tài liệu docs/buoi10-btvn.md	10%	Đủ 4 mục yêu cầu; sơ đồ cây khớp thực tế mã nguồn.	Thiếu ≥ 2 mục; sơ đồ sai hoặc không có.
Vấn đáp cá nhân	10%	Giải thích được vì sao lifting state ở chỗ đó; đọc được luồng dữ liệu trên DevTools.	Không giải thích được lý do; trả lời "AI viết cho em".

LAB 5 BONUS — REFACTOR MỘT TRANG CỦA ĐỒ ÁN NHÓM

Thời gian: 60+ phút (làm ở nhà, theo nhóm đồ án) | **Mức độ:** NÂNG CAO — BONUS +1.0 điểm chuyên cần cả nhóm

Vì sao đây là bonus quan trọng

Từ Bài 8, các nhóm đã bắt đầu dựng phôi đồ án nhóm. Nhiều nhóm có xu hướng đặt tất cả state vào App.tsx cho "tiện" — nhưng đến Bài 17 khi phải tích hợp TanStack Query, việc này sẽ khiến toàn bộ ứng dụng kết xuất lại mỗi lần gõ phím. Lab 5 buộc mỗi nhóm phải trả một "món nợ kỹ thuật" **NGAY BÂY GIỜ**, trước khi món nợ đó trở nên đắt đỏ.

Cách làm

Mỗi nhóm nộp CHUNG một PR duy nhất, nhưng lịch sử commit phải cho thấy đóng góp cá nhân rõ ràng.

► Bước 1. Chọn trang cần refactor

Ngồi lại 15 phút để cả nhóm cùng thống nhất chọn 1 trang. Ưu tiên trang thỏa cả 3 điều kiện:

- Có ít nhất 4 thành phần con.
- Có ít nhất 2 nguồn state khác nhau (từ khóa, bộ lọc, phân trang, biểu mẫu...).
- Hiện tại đang "gộp" tất cả trong một file duy nhất (>80 dòng).

► Bước 2. Chụp ảnh "trước" — commit đầu tiên

```bash
# Đứng trên nhánh chính của nhóm
git checkout main
git pull

# Tạo nhánh refactor
git checkout -b feature/buoi10-lab5-refactor

# Vẽ sơ đồ cây HIỆN TẠI (dùng excalidraw hoặc bút giấy chụp ảnh)
# Lưu vào docs/buoi10-lab5/before.png

# Commit ảnh trước tiên — đây là "state 0"
git add docs/buoi10-lab5/before.png
git commit -m "buoi10-lab5: chup so do TRUOC refactor"
```

► Bước 3. Refactor tách container/view — mỗi bước một commit

QUY TẮC: mỗi bước dưới đây phải là một commit riêng để giảng viên xem được luồng suy nghĩ của cả nhóm. Ai làm commit nào cũng ghi rõ người đó qua git author.

Commit	Người phụ trách gợi ý	Nội dung	Yêu cầu cụ thể
Commit 1	Trưởng nhóm	Vẽ sơ đồ MỤC TIÊU vào docs/buoi10-lab5/target.png	Chỉ ra vị trí đặt state, ai là container, ai là view.

Commit	Người phụ trách gợi ý	Nội dung	Yêu cầu cụ thể
Commit 2	Người phụ trách UI	Tách các presentational component vào file riêng	Chưa cần đổi logic, chỉ dời từng khối JSX ra file mới.
Commit 3	Người phụ trách state	Nâng state lên cha chung, truyền value + callback	Xóa mọi useState "thừa" ở con; con chỉ nhận props.
Commit 4	Người phụ trách kiến trúc	Chuyển logic sang custom hook useXxx()	Container biến mất; view gọi thẳng hook.
Commit 5	Cả nhóm ngồi review	Cập nhật README của nhóm về kiến trúc mới	Ghi rõ "trước / sau" và lý do chọn mẫu.

► Bước 4. Kiểm chứng và mở PR

bash

```
# Kiểm tra 5 commit theo đúng người
git log --pretty="%h %an %s" -5

# Chạy dự án đảm bảo chức năng không bị hỏng
npm run dev

# Đẩy lên GitHub và mở PR
git push origin feature/buoi10-lab5-refactor
# Mở PR, tiêu đề: "Bài 10 Lab 5 — Refactor trang <tên trang>"
# Mô tả PR: gắn ảnh before.png và target.png cạnh nhau
```

CHECKPOINT — Lab 5 BONUS

Chỉ chuyển sang lab tiếp theo khi toàn bộ các mục dưới đây đạt:

- [ ] Trang được chọn có ≥ 4 thành phần con và ≥ 2 nguồn state.
- [ ] Có 5 commit tương ứng 5 bước ở bước 3; git log --pretty cho thấy đủ 5 dòng.
- [ ] Ít nhất 3 người trong nhóm là author của commit (không dồn hết cho 1 người).
- [ ] Chức năng của trang chạy y hệt trước refactor (behavior-preserving refactor).
- [ ] PR có ảnh before + target được đính kèm.
- [ ] README của nhóm cập nhật với sơ đồ cây mới.

Điểm cộng — cả nhóm được +1.0 chuyên cần nếu đạt

Tiêu chí đạt: 6/6 mục checkpoint trên đều tick xanh.

Không có điểm trừ nào nếu không làm — đây thuần túy là bonus.

Điểm chỉ được cộng khi giảng viên hỏi ngẫu nhiên 1 thành viên trong nhóm và người đó giải thích được lý do chọn kiến trúc mới.

BẢNG TEST CASE TỰ KIỂM TRA — TRƯỚC KHI RỜI LỚP

Trước khi ra khỏi phòng học, hãy tự đánh dấu tick vào bảng dưới. Bảng này là cách nhanh nhất để biết mình có sẵn sàng cho kiểm tra giữa kỳ ở Bài 11 hay chưa.

#	Test case	Thao tác kiểm tra	Kết quả mong đợi	Kết quả thực tế
1	Lifting state chạy đúng	Kéo min slider ở TourListPage	TourGrid lọc theo giá; số tour hiện đổi	[ ]
2	PriceFilter không có state riêng	Bấm PriceFilter trong React DevTools	Mục Hooks trống (KHÔNG có State)	[ ]
3	TourListView không có state	Bấm TourListView trong DevTools	Mục Hooks trống	[ ]
4	Container giữ đủ state	Bấm TourListContainer trong DevTools	Có 3 hook State (min, max, query)	[ ]
5	SearchBox tìm không phân biệt hoa/thường	Gõ "TAM giang" vào SearchBox	Đầm phá Tam Giang hiển thị	[ ]
6	Empty state hoạt động	Kéo min = max = 999999	Hiển thị thông báo "Không có tour"	[ ]
7	PageLayout tái sử dụng được	Xem file src/components/PageLayout.tsx	Không import bất cứ gì từ features/tours	[ ]
8	Đã đẩy 3 nhánh lên GitHub	Vào trang repo GitHub	Thấy feature/buoi10-lab1, lab2, lab3	[ ]
9	Không có lỗi console	Mở F12 tab Console	Không có dòng đỏ (lỗi) nào	[ ]
10	TypeScript sạch	Chạy npx tsc --noEmit	Không in ra dòng lỗi nào	[ ]

BẢNG TỰ ĐÁNH GIÁ BÀI 10

Đánh dấu chân thật vào từng kỹ năng. Bảng này không nộp cho giảng viên — chỉ để bạn biết mình cần ôn thêm gì trước Bài 11.

Kỹ năng	Chưa	Cơ bản	Thành thạo	Ghi chú
Vẽ được cây thành phần của một trang React	[ ]	[ ]	[ ]	
Đọc được cây thành phần trong React DevTools	[ ]	[ ]	[ ]	
Nhận diện được state đang đặt sai chỗ	[ ]	[ ]	[ ]	
Nâng state lên cha chung mà không phá chức năng	[ ]	[ ]	[ ]	
Truyền callback từ cha xuống con để cập nhật state	[ ]	[ ]	[ ]	
Tách một thành phần gộp thành Container + View	[ ]	[ ]	[ ]	
Viết custom hook thay cho Container	[ ]	[ ]	[ ]	
Sử dụng children và named slots đúng cách	[ ]	[ ]	[ ]	
Commit theo mẫu buoi10-labX: <mô tả> và push GitHub	[ ]	[ ]	[ ]	
Giải thích được lý do chọn từng mẫu cho người khác	[ ]	[ ]	[ ]	

Chưa đạt ≥ 3 mục "Cơ bản"?

Đừng bỏ qua. Trước Bài 11 (có kiểm tra giữa kỳ), làm 3 việc sau:

1. Đọc lại giáo án lý thuyết Bài 10 mục V, VII, VIII.
2. Xem lại lịch sử commit của Lab 1 và Lab 2 — chú ý diff giữa các commit.
3. Nhắn Zalo/Teams cho một bạn đã đạt "Thành thạo" ít nhất 8/10 mục để nhờ giảng bài lại — cách học này hiệu quả hơn xem YouTube.

TÓM TẮT NHỮNG GÌ ĐÃ HỌC TRONG BÀI 10

Bốn ý cần khắc cốt ghi tâm

1. Cây thành phần là công cụ tư duy trước khi viết mã. Vẽ cây trước, xác định vị trí state, rồi mới gõ.
2. State phải nằm ở CHA CHUNG GẦN NHẤT của tất cả các thành phần cần dùng nó — không cao hơn, không thấp hơn.
3. Presentational chỉ nhận props và vẽ. Container chỉ giữ state và logic. Đây là hai vai trò khác nhau, không được lẫn.
4. Custom hook là biến thể "hiện đại" của container — cùng ý tưởng, ít cấu trúc hơn.

Bảng đối chiếu — trước và sau Bài 10

Trước Bài 10 bạn thường...	Sau Bài 10 bạn sẽ...
Đặt useState ở chỗ đầu tiên nghĩ ra được	Vẽ cây, xác định cha chung, đặt state ở đó
Copy giá trị prop vào state khi cần dùng	Đọc thẳng prop, không sao chép vào state
Viết file 200 dòng với fetch + logic + JSX gộp	Tách thành Container / View hoặc dùng custom hook
Truyền props qua 4–5 tầng cho tiện	Nhận ra "prop drilling", cân nhắc composition với children
Không mở React DevTools bao giờ	DevTools luôn mở, dùng để kiểm chứng mọi refactor

HƯỚNG DẪN NỘP BÀI QUA GITHUB

1. Checklist nhánh phải có

Nhánh	Nội dung	Hạn nộp	Bắt buộc?
feature/buoi10-lab1	Kết quả Lab 1 tại lớp	Cuối Bài	Có
feature/buoi10-lab2	Kết quả Lab 2 tại lớp	Cuối Bài	Có
feature/buoi10-lab3	Kết quả Lab 3 tại lớp	Cuối Bài	Có
feature/buoi10-btvn	Kết quả Lab 4 — module di tích	23:59 trước Bài 11	Có
feature/buoi10-lab5-refactor	Lab 5 BONUS (nhóm)	23:59 trước Bài 11	Không (bonus)

2. Quy chuẩn commit message

Định dạng bắt buộc

```
buoi10-<lab>: <hành động ngắn gọn, tiếng Việt không dấu>

# Ví dụ đúng:
buoi10-lab1: nang state gia len TourListPage
buoi10-lab2: tach TourListContainer va TourListView
buoi10-btvn: hoan thien useDiTichList voi 4 TODO

# Ví dụ SAI:
final
fix
xong bai
sua tao ra dc roi ne
```

3. Quy chuẩn Pull Request

- Tiêu đề: "[Bài 10] Họ tên - MSSV - Tên lab" (VD: "[Bài 10] Nguyễn Văn A - 20211234 - Lab 4 di tích").
- Nhánh nguồn: nhánh feature của lab; nhánh đích: main.
- Mô tả PR: mô tả 3 câu (làm gì, khó ở đâu, tự đánh giá bao nhiêu %).
- Gắn label "buoi10" (giảng viên đã tạo sẵn nhãn trên repo).
- Reviewer: @cuong.nguyen (giảng viên sẽ review trong 48h).

4. Kiểm tra trước khi bấm nút "Create pull request"

Đóng cửa sổ PR nếu 1 trong 4 điều dưới đây chưa đạt

npm run dev chạy không lỗi ở máy của bạn.

npx tsc --noEmit không in ra lỗi TypeScript nào.

Đã tick TOÀN BỘ checklist trong CHECKPOINT của lab tương ứng.

File README.md của repo có ghi tên nhánh nộp bài (giảng viên sẽ tìm theo đây).

PREVIEW BÀI 11 — TYPESCRIPT + KIỂM TRA GIỮA KỲ

Bài 11 quan trọng — mốc đánh giá 16% học phần

Nội dung mới: TypeScript cho React (kiểu cho props, state, hook, sự kiện).

Kiểm tra giữa kỳ 45 phút vào 60 phút cuối Bài — làm cá nhân, nộp qua GitHub.

Kịch bản kiểm tra KHÔNG trùng với bài lab của Bài 10 — nhưng dùng CÙNG một bộ kỹ năng (lifting state, tách container/view, kiểu TypeScript).

Cần chuẩn bị gì trước Bài 11

Việc cần làm	Ước tính thời gian	Vì sao quan trọng
Hoàn thành Lab 4 (BTVN)	90–120 phút	Không hoàn thành = không đủ nền để làm kiểm tra giữa kỳ.
Đọc trước "Static type-checking" trên vi.react.dev/learn	15 phút	Bài 11 dạy áp dụng, không dạy lại lý thuyết cơ bản.
Kiểm tra phiên bản TypeScript của lớp	2 phút	Chạy npx tsc --version, phải khớp bản khoá của học phần.
Xem lại giáo án lý thuyết Bài 10 mục XI	10 phút	Bảng 8 lỗi thường gặp — đây là các lỗi hay xuất hiện trong đề kiểm tra.
Ngủ đủ giấc đêm trước	Xuyên đêm	Không đùa — kiểm tra 45 phút cần đầu óc tỉnh táo.

Kết thúc bộ tài liệu thực hành Bài 10. Chúc bạn code vui, git commit sạch, và có nhiều "aha moment" khi mở React DevTools. Hẹn gặp lại ở Bài 11.